RICE UNIVERSITY

# Tensor and Model Parallelism

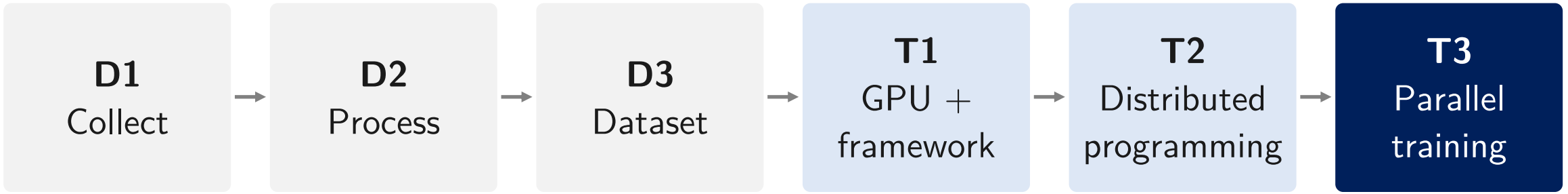
Splitting one layer across GPUs: block matrices, two layouts, and one all-reduce

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 12, Lecture 2

# Where we are in the pipeline




**In our pipeline**

Today: split **one layer** across GPUs. Lab 5 (tensor and pipeline partitions of a toy model) is released after this lecture.

Lecture 1: split the batch → today: split each layer → Week 13: split the layer stack

# Learning goals for today

- **Explain** why one layer can be too big for one GPU, and what tensor parallelism splits.
- **Derive** the column split and the row split of a matrix multiply with block matrices.
- **Build** the column-then-row MLP with exactly **one all-reduce** in forward.
- **Estimate** per-GPU memory, and explain why tensor parallelism stays **inside one server**.

# A puzzle to start

**Two-layer MLP**

$Y = \tanh(X\ A)\ B$

X: one row per example

**Two GPUs**

each holds half of A

and half of B

**How few times must the GPUs communicate to compute Y?**

**Answer**

**Once.** Split A by **columns** and B by **rows**: tanh stays on each GPU, and one all-reduce adds the two partial results.

Source: Shoeybi et al., "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," 2019.

# Recap: two tools from last lecture

- **all-reduce**: each GPU starts with its own vector; every GPU ends with the **sum**.
- Training with Adam: **16 bytes per parameter**, so $16\Psi$ bytes for $\Psi$ parameters.

$$T(S,\, N) \approx 2(N-1)/N \cdot S \;/\; \beta$$

**Lecture 1**

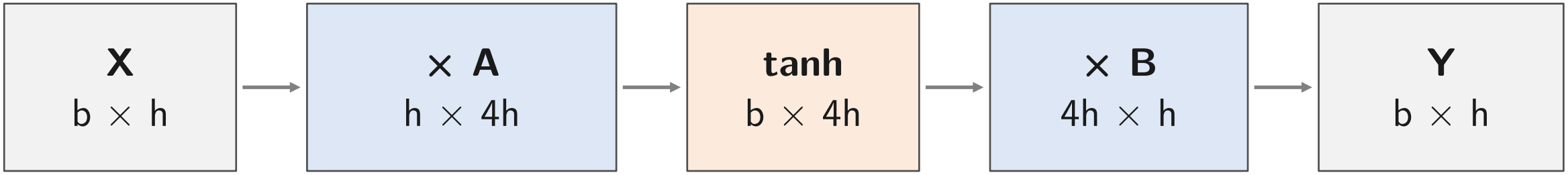
S: bytes per GPU, $\beta$: link bandwidth. For large S the time is set by **bandwidth**.

Part 1

# Why split one layer?

What if a single layer is too big for one GPU?

# Meet a wide MLP block




- **b**: rows of X — examples processed together.
- **h**: the width of each row; the hidden layer is **4h** wide.

**Key idea**

Same pattern as our course MLP — multiply, tanh, multiply — just much **wider**.

Source: Shoeybi et al., "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," 2019.

# How big is one wide block?

$$\Psi = (h \times 4h) + (4h \times h) = 8h^2$$

$$h = 16{,}384: \quad \Psi \approx 2.1 \times 10^9 \text{ parameters}$$

$$\text{training state:} \quad 16\Psi \approx 34 \text{ GB}$$

**Watch out**

One block's training state is almost **half of an 80 GB GPU** — and a model stacks dozens.

Source: H100 datasheet (80 GB); the block size is our example.

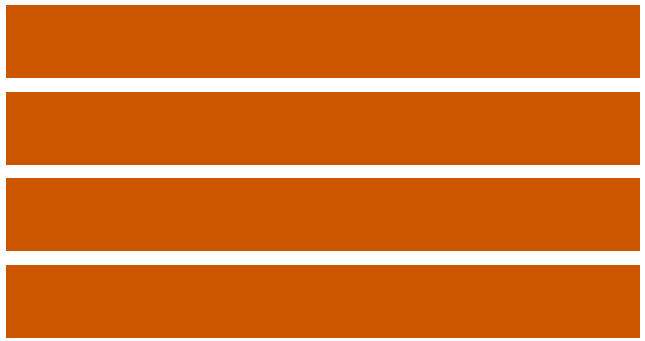
# Why not more data parallelism?

- **Plain DP**: every GPU holds **every whole layer**.
- **ZeRO-3 / FSDP**: stores 1/N, but **gathers the full layer** to compute it.
- Both: each layer's arithmetic runs on **one GPU** per example.

**Key idea**

Tensor parallelism splits the **weights and the work inside each layer**.

# Three ways to split training

| Data | Tensor | Pipeline |
|---|---|---|
|  |  |  |
| **split the batch** | **split each layer's matrices** | **split the layer stack** |
| every GPU: whole model | every GPU: part of every layer | every GPU: some whole layers |

**Key idea**

Today: **tensor** parallelism. Week 13: **pipeline**, and all three together (3D).

# Check yourself

Two minutes with a neighbour.

- A wide block with $h = 8{,}192$: how many parameters, and how many GB of training state?
- Why does adding data-parallel GPUs not make one layer smaller on each GPU?
- What does each GPU hold under data, tensor, and pipeline parallelism?

Part 2

# Splitting a matrix multiply

Columns or rows: what does each GPU compute?

# Block matrices: the one identity we need

$$X\text{: } m \times k, \quad A\text{: } k \times n, \quad Y = \mathrm{X}\mathrm{A}\text{: } m \times n$$

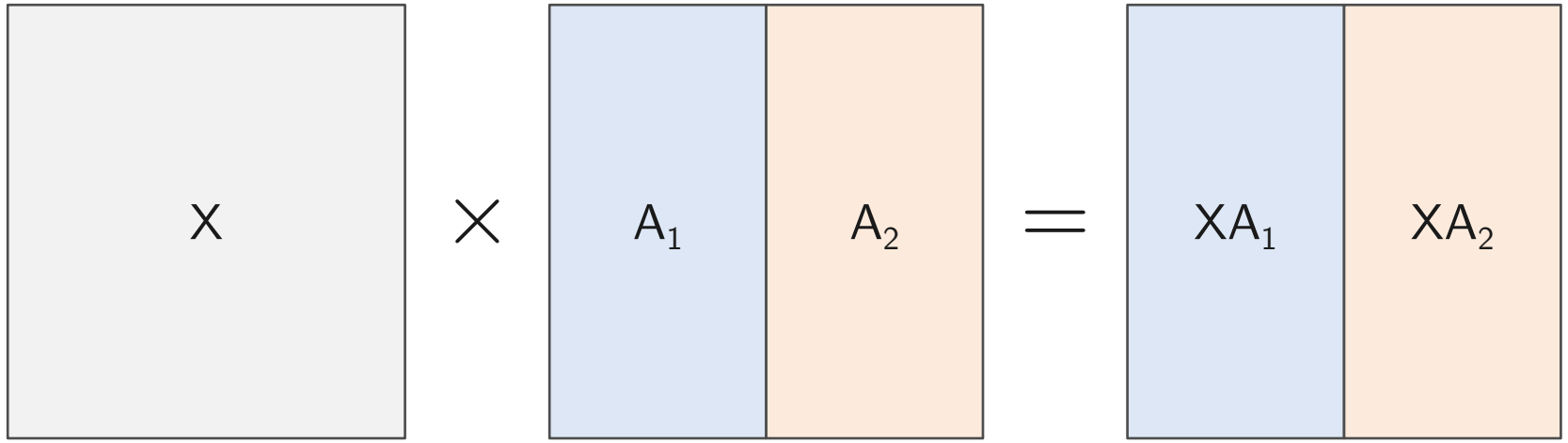
$$\text{columns: } \; A = [A_1 \;\; A_2] \quad \Rightarrow \quad \mathrm{X}\mathrm{A} = [\mathrm{X}\mathrm{A}_1 \quad \mathrm{X}\mathrm{A}_2]$$

$$\text{rows: } \; A = [A_1 \; ; \; A_2], \;\; X = [X_1 \;\; X_2] \quad \Rightarrow \quad \mathrm{X}\mathrm{A} = X_1 A_1 + X_2 A_2$$

**Key idea**

Column split $\rightarrow$ **pieces of Y**. Row split $\rightarrow$ **partial sums of Y**. Everything else follows.

# Column split: each GPU owns a slice of Y



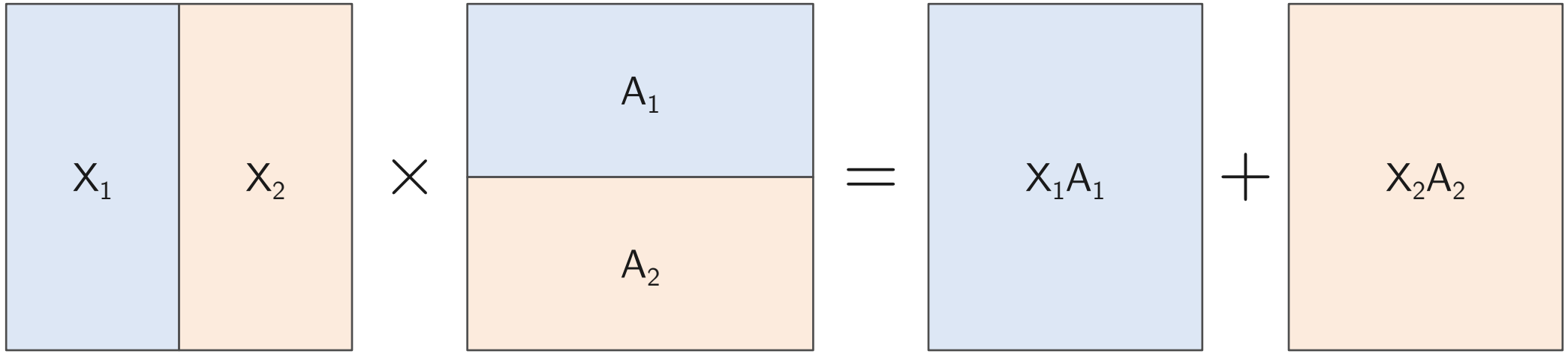

**No communication** to compute the pieces.

Each GPU holds **1/N** of A and of Y.

**Memory**

Need the full Y on every GPU? **All-gather** the slices. Often the next layer does not.

# Row split: each GPU owns a partial sum




X by columns, A by rows

GPU 0 and GPU 1: one full-size term each

**Memory**

Every GPU needs the **sum** $\rightarrow$ one **all-reduce** of a full-size Y. The input needs nothing if it already arrives split by columns.

# The two layouts side by side

| | **Column split of A** | **Row split of A** |
|---|---|---|
| Input X | whole, on every GPU | split by columns |
| Output Y | split by columns | partial sums |
| Communication | none | one all-reduce |

**Key idea**

The column split **outputs** exactly what the row split **expects as input**. Chain them.

# Where can tanh go?

$$\text{column split:}\quad \tanh([\mathrm{XA}_1 \quad \mathrm{XA}_2]) = [\tanh(\mathrm{XA}_1) \quad \tanh(\mathrm{XA}_2)]$$

$$\text{row split:}\quad \tanh(X_1A_1 + X_2A_2) \neq \tanh(X_1A_1) + \tanh(X_2A_2)$$

- Elementwise functions (tanh, GeLU) act **entry by entry**: fine on column slices.
- On partial sums they need the **full sum first** $\rightarrow$ an all-reduce before them.

**Watch out**

Put tanh **after a column split**, never on the partial sums of a row split.

Source: Shoeybi et al., "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," 2019.

# Check yourself

Two minutes with a neighbour.

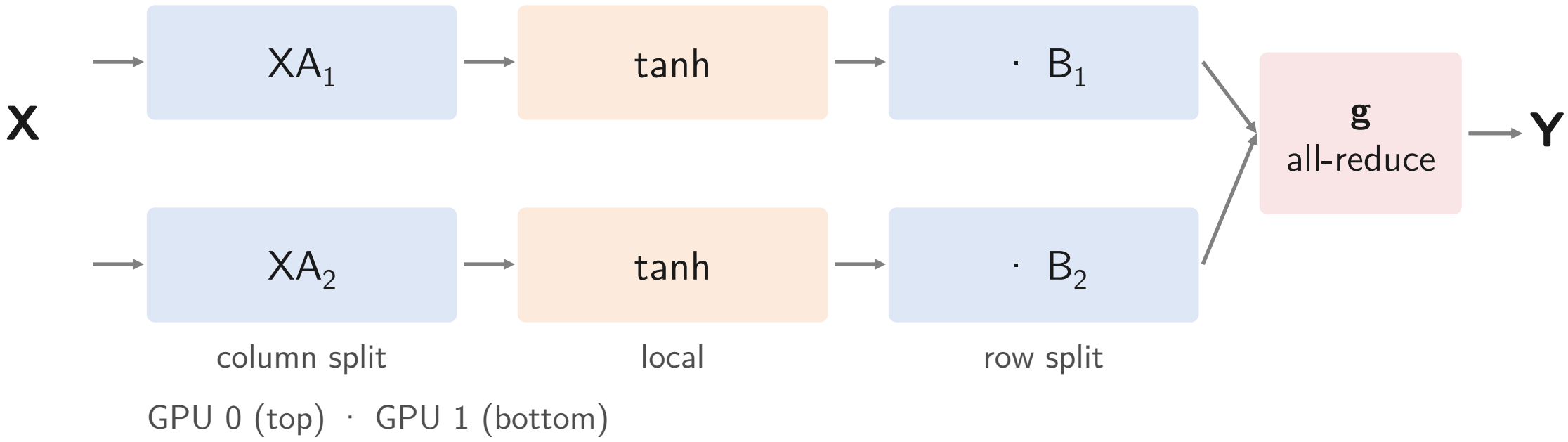
- Which split of A lets each GPU compute its part of Y with **no** communication?
- Why can tanh not be applied to the partial sums of a row split?
- A row split needs X split by columns. Which layer produces X in that layout?

Part 3

# Column, then row

One all-reduce per block — what does it cost, and what fits?

# Column, then row: one all-reduce




**Key idea**

Forward: **one all-reduce** (g) of the h-wide output. The 4h-wide hidden layer never moves.

Source: Shoeybi et al., "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," 2019.

# The other order costs more

| Order | Communication in forward |
|---|---|
| column, then row | one all-reduce of Y $(b \times h)$ |
| row, then column | all-reduce of the hidden layer $(b \times 4h)$, then all-gather of Y |

**Answer to the puzzle**

Column, then row: tanh stays local and only the **narrow** output is communicated.

# Training adds one more: f and g

| Operator | Where | Forward | Backward |
|---|---|---|---|
| f | before the column split | nothing | all-reduce |
| g | after the row split | all-reduce | nothing |

Autograd inserts the backward all-reduce for us; we never derive it.

**Memory**

Per block and training step: **one all-reduce in forward, one in backward**, each b $\times$ h.

Source: Shoeybi et al., "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," 2019.

# How big is the message?

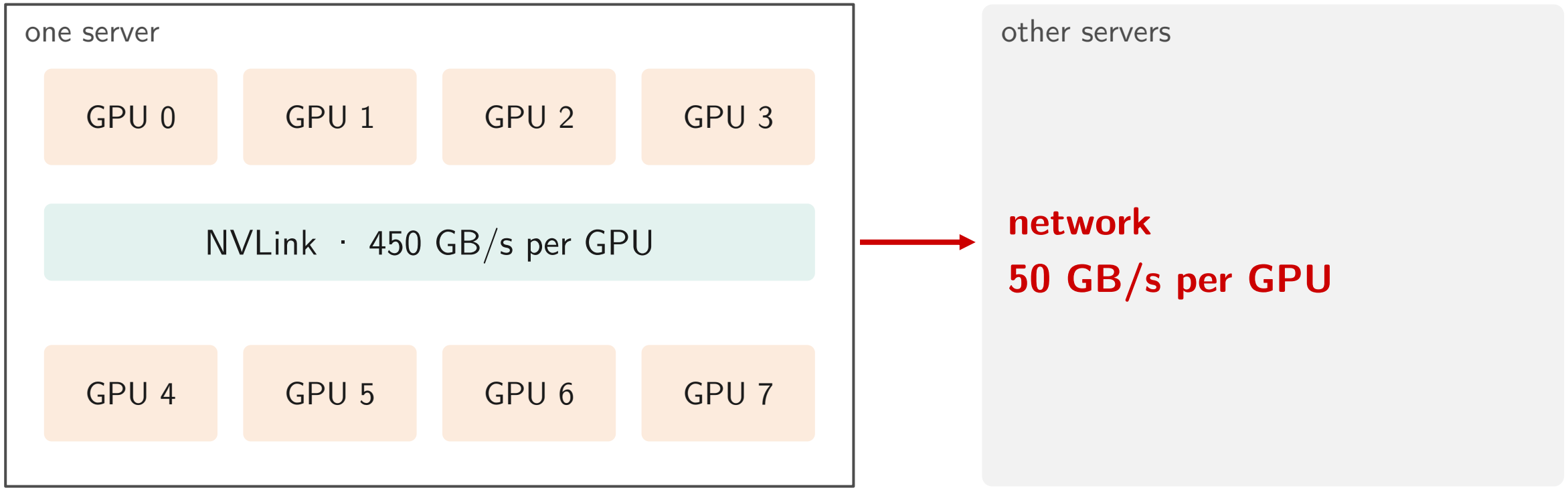
$$\text{message} = Y\text{:} \quad S = 2 \cdot b \cdot h \text{ bytes} \quad (\text{bf16})$$

$$b = 8{,}192 \text{ rows}, \quad h = 16{,}384\text{:} \quad S \approx 268 \text{ MB}$$

**Memory**

Tensor parallelism sends **activations** (b $\times$ h), not parameters — and it does so in **every block**.

# Inside a server versus between servers

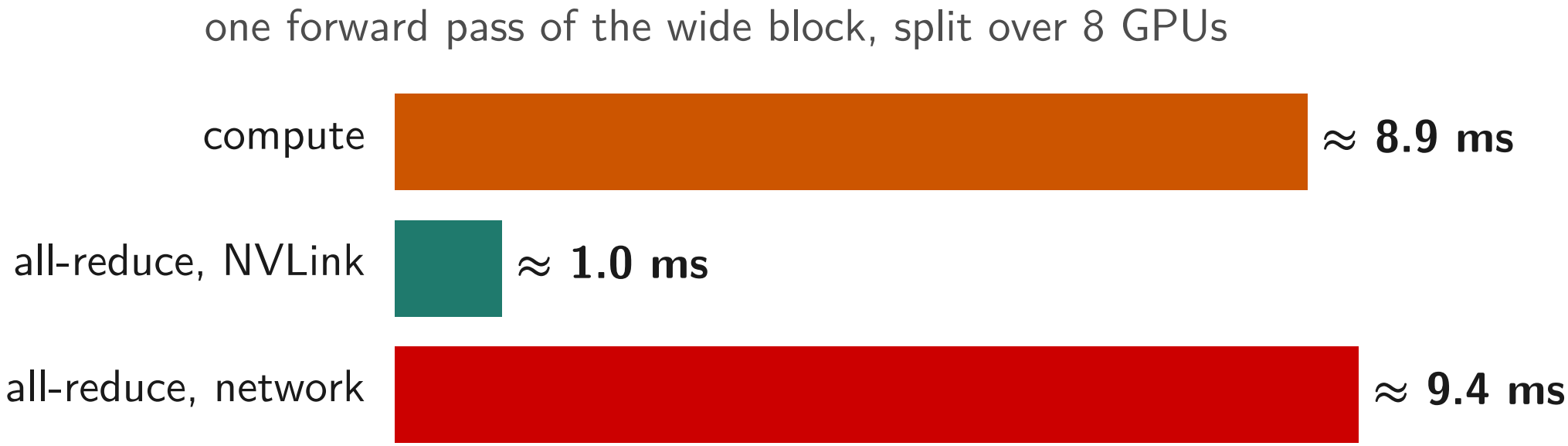



**Memory**

Leaving the server cuts the bandwidth per GPU by **9×**.

Source: NVIDIA DGX H100 User Guide (8× ConnectX-7, 400 Gb/s per GPU); H100 datasheet (NVLink 900 GB/s bidirectional). Per direction.

# Why tensor parallelism stays in one server




**Watch out**

Over NVLink the all-reduce is small next to compute; over the network it is **longer**. Keep **N ≤ GPUs per server**.

Source: H100 datasheet; DGX H100 User Guide; 50% of bf16 peak assumed; Narayanan et al., SC 2021.

# Per-GPU memory: a simple fraction

$$\text{training state per GPU} = 16\Psi \;/\; N$$

$$\text{wide block, } N = 8: \quad 34 \text{ GB} \;\rightarrow\; 4.3 \text{ GB}$$

**Key idea**

Same **1/N** as ZeRO-3 — but no GPU ever holds the **whole layer**.

# What is not split stays whole

$$\text{per GPU} = \text{replicated} + \text{split} \;/\; N$$

| Course MLP | Layout | Parameters |
|---|---|---|
| W1, b1 | column split | 8,256 |
| W2 | row split | 20,608 |
| embedding, b2 | replicated | 5,474 |

**Watch out**

N = 2: **19,906** parameters per GPU, not half of 34,338. Add b2 **once**, after the all-reduce.

# Check yourself

Two minutes with a neighbour.

- Where is the one forward all-reduce in the column-then-row block, and what shape does it carry?
- Why is splitting one block over 16 GPUs in two servers a bad idea?
- On 4 GPUs, why does our MLP not shrink to 34,338 / 4 parameters per GPU?

IN-CLASS BUILD

# A 2-way tensor-parallel MLP

Work in pairs. Part A: code the forward and check it. Part B: per-GPU memory.

# Exercise A: column-then-row forward

```
A_r = A.chunk(2, dim=___)[r]      # (1)
b_r = b.chunk(2)[r]
B_r = B.chunk(2, dim=___)[r]      # (2)
y = torch.tanh(x @ A_r + b_r) @ B_r
dist.all_reduce(y)                # (3) why?
y = y + c                         # (4) why here?
ref = torch.tanh(x @ A + b) @ B + c
torch.testing.assert_close(y, ref)
```

**QUESTIONS**

Fill in (1)–(2); explain (3)–(4).

What are the shapes of `A_r`, `B_r` and `y` on each rank?

Every rank builds the same A ($128 \times 64$), B ($64 \times 322$), b, c, x from one seed; r is this rank.

**STRETCH**

Repeat on 4 ranks (chunk into 4): what are the shapes, and does it still match?

# Exercise B: per-GPU memory

| Quantity | N = 2 | N = 4 |
|---|---|---|
| course MLP: parameters per GPU | | |
| course MLP: tanh output per GPU (bytes) | | |
| wide block, h = 8,192: training state per GPU | | |

**GIVEN**

Course MLP: 34,338 parameters, 5,474 of them replicated; batch 64, hidden 64, FP32 = 4 bytes. Wide block: $\Psi = 8h^2$, 16 bytes per parameter, GB = $10^9$ bytes.

# Solutions A: the forward check

- (1) `dim=1`: `A_r` is $128 \times 32$ (columns).
- (2) `dim=0`: `B_r` is $32 \times 322$ (matching rows).
- (3) Each y is a **partial sum** ($64 \times 322$): the all-reduce is g.
- (4) c is on every rank: add it **once**, after the sum.

**Check**

Stretch: on 4 ranks the pieces shrink, y stays $64 \times 322$ — and it still matches.

# Solutions B: the memory count

| Quantity | N = 2 | N = 4 |
|---|---|---|
| course MLP: parameters per GPU | 19,906 | 12,690 |
| course MLP: tanh output per GPU (bytes) | 8,192 | 4,096 |
| wide block, h = 8,192: training state per GPU | 4.3 GB | 2.1 GB |

**Check**

Split parts divide by N; **replicated** parts do not — so the total is never exactly 1/N.

# How to test it

- Run on **CPU** first (gloo backend), then on GPUs.
- Compare with a **single-process** reference using `assert_close`.
- Repeat on **4 ranks**.
- Delete the all-reduce: the test must **fail**.

**In our pipeline**

Lab 5: tensor and pipeline partitions of a toy model, bubbles, and a 3D layout.

# Key takeaways

- Column split of A $\rightarrow$ **slices of Y**, no communication; row split $\rightarrow$ **partial sums**, all-reduce.
- tanh acts entry by entry: it is local on **column** slices, so put it between column and row.
- Column, then row: **one all-reduce** forward (g), one backward (f, inserted by autograd).
- It carries **activations** (b $\times$ h) in every block: keep it on **NVLink, inside one server**.
- Per GPU: split parts shrink as **1/N**; replicated parts do not.

# Next time, and readings

| When | What |
|---|---|
| After this lecture | Lab 5 released: tensor and pipeline partitions, bubbles, 3D layout |
| Week 13, Lecture 1 | Pipeline and 3D parallelism |
| Week 13, Lecture 2 | Diagnosing distributed systems |

**READ BEFORE WEEK 13**

Shoeybi et al., "Megatron-LM," 2019 — Section 3 (the MLP split).

Narayanan et al., "Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM," SC 2021.